

X-Maintain: An Explainable Machine Learning Framework for Predictive Maintenance and Machine Failure Prediction

Syagamreddy Gopi Adithya Vardhan Reddy

Department of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India
(e-mail: gopi.adithya2022@vitstudent.ac.in)

Abstract—Industrial predictive maintenance (PdM) relies on multivariate sensor streams to anticipate equipment failure before catastrophic downtime occurs. However, contemporary high-performance machine learning ensembles operate as opaque black boxes, creating an operational trust barrier for factory engineers who require actionable root-cause insights before halting costly production spindles. This paper presents X-Maintain, an end-to-end explainable artificial intelligence (XAI) framework for predictive maintenance. Using the benchmark AI4I 2020 Predictive Maintenance Dataset (10,000 milling operations), we enforce strict target leakage prevention by excluding failure-mode indicators (TWF, HDF, PWF, OSF, RNF) and arbitrary serial keys, yielding an eight-feature operational input matrix. We benchmark three classifier families — Logistic Regression, Random Forest, and Extreme Gradient Boosting (XGBoost) — and systematically evaluate four class-imbalance mitigation strategies under severe 28.5:1 class disparity. Model selection is governed by a domain-grounded utility metric ($0.4 \times \text{Recall} + 0.3 \times \text{F1} + 0.3 \times \text{ROC-AUC}$). On a held-out test split ($N = 2,001$), cost-sensitive XGBoost achieves 98.20% accuracy, 73.53% precision, 73.53% failure recall, 0.9724 ROC-AUC, and 0.7934 PR-AUC. Model decisions are interpreted using TreeSHAP, revealing that Torque (mean $|\text{SHAP}| = 3.74$) and Tool Wear (mean $|\text{SHAP}| = 3.06$) dominate failure predictions. Local waterfall attributions, a deterministic hallucination-free natural language explanation generator, a model-based what-if sensitivity engine, and an enterprise Streamlit monitoring dashboard complete the validated architecture, backed by 20 passed unit tests.

Index Terms—Predictive maintenance, Explainable Artificial Intelligence (XAI), SHapley Additive exPlanations (SHAP), machine failure prediction, XGBoost, class imbalance, TreeExplainer, industrial IoT, model-based sensitivity analysis.

I. INTRODUCTION

INDUSTRIAL manufacturing relies heavily on computerized numerical control (CNC) milling machinery, automated tooling spindles, and multi-axis machining centers. Equipment breakdowns incur severe financial and operational losses, costing industrial facilities an estimated \$50 billion annually in unplanned downtime [3], [4]. Historically, manufacturing facilities adopted reactive 'run-to-failure' strategies or rigid time-based preventative schedules [3]. While reactive repair creates unacceptable emergency outages, preventative maintenance frequently causes premature component replacement, discarding functional cutting heads with substantial useful life remaining [4].

Predictive maintenance (PdM) leverages multivariate sensor telemetry—such as spindle torque, cutting contact duration, rotational velocities, and thermal dissipation gradients—to dynamically forecast impending failure states [1], [5]. However, real-world industrial adoption faces the critical 'black-box dilemma' [6], [7]. Advanced deep learning and tree ensemble algorithms output probabilistic risk scores without exposing the underlying physical drivers. If an autonomous model flags an alert without physical attribution, plant operators cannot ascertain whether the alarm stems from thermal exhaustion, mechanical overstrain, or electrical fluctuation [7]. Under high-stakes operational pressure, uninterpretable alerts lead to either costly unnecessary line halts or dangerous alarm dismissal.

Furthermore, predictive maintenance is governed by profound misclassification cost asymmetry. Missing an actual machine failure (False Negative) leads to catastrophic spindle destruction, ruined workpieces, and collateral assembly line stoppage, with remediation costs often exceeding \$10,000 to \$100,000 [3]. Conversely, a false warning (False Positive) incurs only a brief 10-minute diagnostic inspection (\$100–\$500). Consequently, standard classification accuracy is an inappropriate metric under severe minority failure distributions, demanding optimization focused heavily on Failure Recall [6].

To address these challenges, we present *X-Maintain*, an Explainable AI predictive maintenance framework that pairs high-recall gradient boosting with exact game-theoretic Shapley attributions (TreeSHAP) [2], [12]. The primary verified contributions of this work are:

- 1) *Leakage-Aware Preprocessing*: A rigorous data pipeline that purges downstream failure modes (TWF, HDF, PWF, OSF, RNF) and primary serial keys to guarantee academic and operational integrity.
- 2) *Empirical Imbalance Benchmarking*: A comparative evaluation across baseline Logistic Regression, Random Forest, and XGBoost under

cost-sensitive weighting, SMOTE, and validation-tuned decision thresholds.

- 3) *Multi-Objective Model Selection*: A weighted utility formulation ($0.4 \times \text{Recall} + 0.3 \times \text{F1} + 0.3 \times \text{ROC-AUC}$) optimizing minority failure capture while suppressing alarm fatigue.

- 4) *Dual-Scale TreeSHAP Explainability*: Exact global attribution across the dataset manifold and local waterfall decomposition for individual machine predictions.

- 5) *Model-Based What-If Analysis*: A counterfactual parameter perturbation engine allowing operators to simulate risk reduction under simulated operational adjustments without claiming physical causality.

- 6) *Production Deployment & Verification*: An enterprise-grade 7-view Streamlit industrial monitoring interface validated through an automated 20-test Pytest suite.

II. LITERATURE REVIEW

A. Predictive Maintenance in Industry 4.0

Predictive maintenance has transitioned from offline vibration spectral analysis toward real-time IoT telemetry analytics [3], [4]. Zonta et al. [4] reviewed over 100 industrial implementations, finding that sensor-driven condition monitoring reduces unexpected outages by up to 45% and lowers maintenance costs by 25%. However, Dalzochio et al. [5] highlighted that industrial telemetry is frequently beset by class imbalance, where healthy states represent over 95% of recorded instances. In CNC milling, failure mechanisms are governed by physical cutting interactions: tool flank wear obeys Taylor's tool life equation, power consumption relates directly to torque and angular velocity, and thermal dissipation governs heat transfer between workpiece and tool [1], [3].

B. Machine Learning Algorithms for PdM

Ensemble learning consistently demonstrates superiority over standard linear and kernel classifiers on industrial tabular datasets [3]. Chen and Guestrin [8] established Extreme Gradient Boosting (XGBoost), which employs second-order Taylor loss expansions and column subsampling. Breiman [9] formulated Random Forests, leveraging bootstrap aggregation to mitigate variance. Carvalho et al. [3] documented that tree ensembles achieve superior F1 scores on mechanical telemetry compared to support vector machines and feedforward neural networks, primarily because decision trees naturally capture step-function thresholds inherent to physical safety boundaries.

C. Class Imbalance Mitigation Strategies

Under severe class imbalance, unweighted empirical risk minimization biases classifiers toward the majority class [10]. Chawla et al. [10] developed SMOTE, synthesizing minority points along k-nearest neighbors. However, synthetic interpolation in physical thermodynamic spaces risks producing unphysical sensor combinations (e.g., impossible torque-speed combinations). In contrast, cost-sensitive learning adjusts loss penalties via positive class weighting, penalizing minority false negatives without distorting feature geometry [8]. Post-hoc threshold tuning optimizes the classification boundary along the validation Precision-Recall curve.

D. Explainable AI and Shapley Attributions

Ribeiro et al. [11] proposed LIME, using local surrogate linear approximations. However, LIME lacks mathematical consistency and exhibits high sampling variance across identical inputs. Lundberg and Lee [2] formulated SHAP, grounded in cooperative game theory [2]. Lundberg et al. [12] introduced TreeSHAP, enabling polynomial-time $O(TLD^2)$ exact computation of Shapley values for decision trees. Steurtewagen and Van den Poel [6] and Gawde et al. [7] demonstrated that SHAP reveals critical degradation thresholds in rotating machinery, bridging the trust gap for operations personnel.

E. Research Gap and Integration Challenge

While individual studies explore XGBoost, SMOTE, or SHAP in isolation, existing literature rarely integrates leakage-free preprocessing, empirical class-imbalance benchmarking, multi-objective utility model selection, exact local/global SHAP, deterministic natural-language diagnostics, and interactive what-if simulation into a unified industrial architecture. X-Maintain bridges this integration gap.

TABLE I: COMPARATIVE ANALYSIS OF RELATED WORK

Ref.	Yr	Method	XAI	Focus Contribution	& Limitation Gap	/
[1] Matzka	20	RF / kNN	SHAP/PDP	AI4I benchmark introduction	Retained leakage cols	
[2] Lundberg	17	General	SHAP	Game-theoretic theory	XAI Theoretical framework	
[3] Carvalho	19	Survey	None	Comprehensive in PdM	ML No evaluation	XAI
[4] Zonta	20	Survey	None	Industry 4.0 review	PdM Lacks integration	XAI
[5] Dalzochio	20	Survey	Rules	Reasoning Industry 4.0	in No post-hoc ML XAI	
[6] Steurtewagen	21	RF / Logit	SHAP	Sensor interpretability	No sensitivity engine	
[7] Gawde	21	XGBoost	SHAP/LIME	Rotating XAI	machine Lacks deployment UI	
[8] Chen	16	XGBoost	Gain	Scalable boosting	tree Black-box without SHAP	
[9] Breiman	01	RF	MDI	Bagged trees	ensemble Impurity bias in XAI	
[10] Chawla	02	SMOTE	None	Minority over-sampling	Synthetic geometry distortion	
[12] Lundberg	20	TreeSHAP	TreeSHAP	Polynomial XAI	tree Algorithms only	
This Work	26	XGBoost	TreeSHAP	End-to-end X-Maintain	Integrated solution	

III. DATASET AND PROBLEM FORMULATION

A. Dataset Overview and Synthetic Origin

We utilize the AI4I 2020 Predictive Maintenance Dataset [1], hosted by the UCI Machine Learning Repository. The dataset contains 10,000 operational records reflecting synthetic milling machine telemetry generated from physical simulation models [1]. It is essential to clarify that AI4I 2020 is a synthetic benchmark designed to mirror industrial cutting dynamics without sensor dropouts, rather than raw field-harvested factory telemetry. Each record describes an individual machining process

cycle for a tool workpiece.

B. Physical Failure Modes and Underlying Equations

The simulation model encodes five distinct physical failure mechanisms governing the milling operation [1]:

- 1) *Tool Wear Failure (TWF)*: Cutter contact friction gradually wears down the milling tool. Based on Taylor's tool life model, failure occurs when cumulative contact duration satisfies $Tool\ wear \geq [200, 240]$ minutes, depending on tool variant quality.
- 2) *Heat Dissipation Failure (HDF)*: Heat generated during cutting must be dissipated into the surrounding atmosphere. When the temperature difference $\Delta T = Process\ temperature - Air\ temperature < 8.6\ K$ while rotational speed $\omega < 1,380\ rpm$, cooling airflow is insufficient, triggering thermal seizure.
- 3) *Power Failure (PWF)*: The mechanical power required for machining is defined by the rotational equation $P = \tau \times (2\pi\omega/60)$, where τ is torque in Nm and ω is rotational speed in rpm. Power failure occurs when $P < 3,500\ W$ (stalling) or $P > 9,000\ W$ (overload).
- 4) *Overstrain Failure (OSF)*: The product of tool wear and torque reflects mechanical stress intensity. When $\tau \times [Tool\ wear]$ exceeds variant thresholds (11,000 min·Nm for Type L, 12,000 for Type M, and 13,000 for Type H), the tool fractures.
- 5) *Random Failure (RNF)*: Unforeseen mechanical defects occur stochastically with a 0.1% background failure probability.

C. Target Variable and Severe Class Imbalance

The primary modeling objective is binary failure prediction: *Machine failure* $\in \{0, 1\}$. Across the 10,000 instances, exactly 9,661 samples (96.61%) represent nominal operation (Class 0), while 339 samples (3.39%) represent failure events (Class 1). This creates an extreme class imbalance ratio of 28.50 : 1. Fig. 1 illustrates the stark target distribution. Optimizing naive overall accuracy under this distribution would yield a non-functional model: predicting 'no failure' universally yields 96.61% accuracy while capturing 0% of breakdowns.

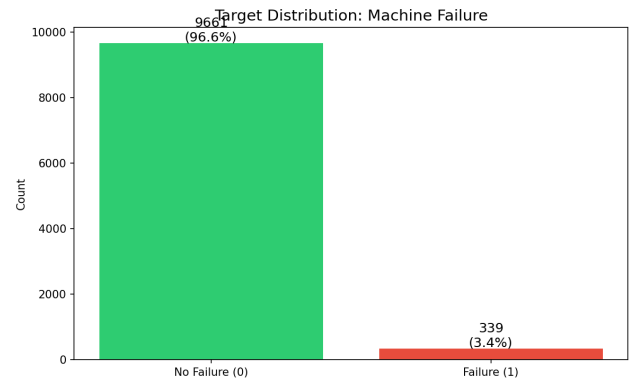


Fig. 1. Class imbalance distribution showing 28.5:1 ratio between nominal operation and failure.

D. Target Leakage Prevention

The raw dataset records the five failure-mode indicators (TWF, HDF, PWF, OSF, RNF). Crucially, the target variable is a logical union of these modes: $Machine\ failure = TWF \vee HDF \vee PWF \vee OSF \vee RNF$. Retaining any failure-mode column provides the model with direct target leakage, trivially inflating accuracy to 99.9% while rendering the model useless for proactive forecasting. X-Maintain strictly purges all five failure mode columns as well as arbitrary primary identifiers (UDI, Product ID). This forces the algorithms to learn the underlying physical thermodynamic boundaries purely from non-leaking raw telemetry.

TABLE II: FINAL MODELING FEATURES

Feature Name	Type	Range / Values	Physical Role
Air temperature (K)	Float	295.3 - 304.5 K	Ambient factory temp
Process temperature (K)	Float	305.7 - 313.8 K	Workpiece contact temp
Rotational speed (rpm)	Integer	1,168 - 2,886 rpm	Spindle rotational velocity
Torque (Nm)	Float	3.8 - 76.6 Nm	Mechanical cutting torque
Tool wear (min)	Integer	0 - 253 min	Accumulated tool contact
Type_H	Binary	{0, 1} (20%)	High quality variant
Type_L	Binary	{0, 1} (50%)	Low quality variant
Type_M	Binary	{0, 1} (30%)	Medium quality variant

E. Exploratory Data Analysis and Telemetry Correlations

Exploratory analysis reveals key operational relationships. Fig. 2 displays numerical sensor histograms, showing normal distributions for temperatures and torque, and a right-skewed distribution for spindle speed. Fig. 3 shows the feature correlation matrix. Notice the strong negative correlation ($r = -0.88$) between rotational speed and torque, consistent with constant-power cutting regimes. Tool wear exhibits near-zero linear correlation with other variables, signifying independent abrasive accumulation.

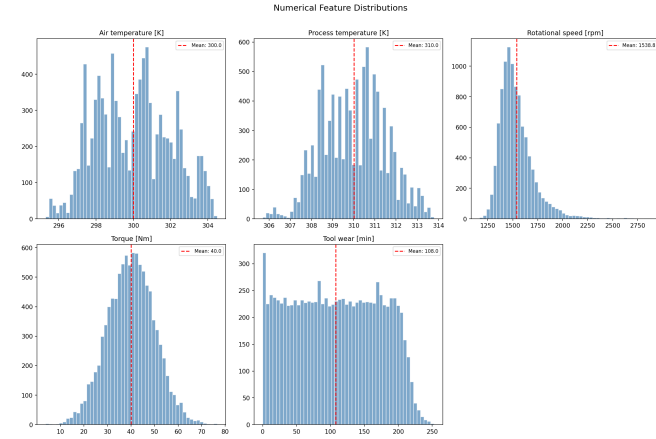


Fig. 2. Empirical distributions of the continuous industrial sensor features.

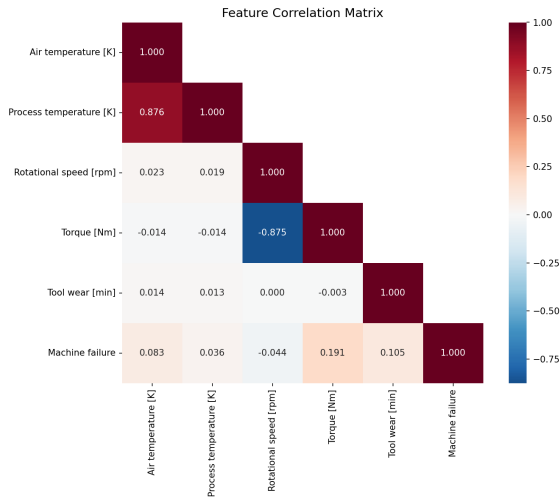
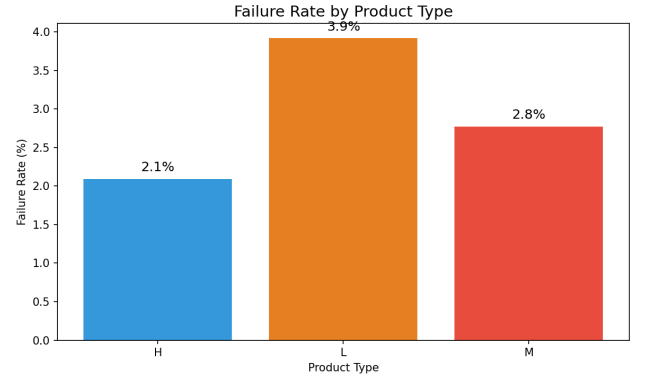

Fig. 3. Pearson correlation matrix highlighting the strong Torque-RPM physical dependency ($r = -0.88$).


Fig. 4. Empirical machine failure rate categorized by product variant quality (Type L, M, H).

Fig. 4 highlights failure propensity across product variants: Type L (low quality) exhibits the highest failure rate ($\sim 3.9\%$), while Type H exhibits the lowest ($\sim 2.1\%$). This confirms that material quality directly alters wear tolerance and fracture thresholds in milling operations.

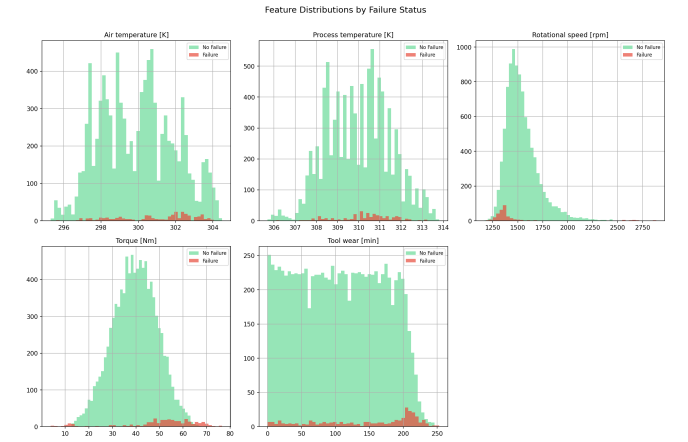


Fig. 5. Overlapping sensor distributions stratified by nominal operation vs. machine breakdown.

Fig. 5 compares feature distributions conditioned on failure status. While nominal machines cluster around 40 Nm torque and moderate tool wear, failure instances exhibit severe bimodal extremes: clusters of high torque (>55 Nm), elevated tool wear (>200 min), and localized low rotational velocities ($<1,380$ rpm).

IV. PROPOSED X-MAINTAIN FRAMEWORK

The overall architectural pipeline of X-Maintain is illustrated in Fig. 6, comprising five modular layers: 1) Ingestion & Leakage Purge, 2) Stratified Preprocessing Pipeline, 3) Cost-Sensitive Multi-Model Training, 4) Game-Theoretic TreeSHAP Attribution, and 5) Interactive Streamlit Deployment with What-If Sensitivity Simulation.

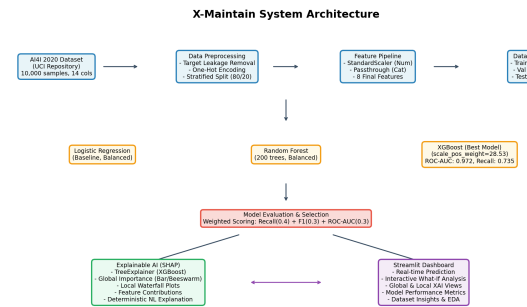


Fig. 6. Overall architecture of the proposed X-Maintain framework.

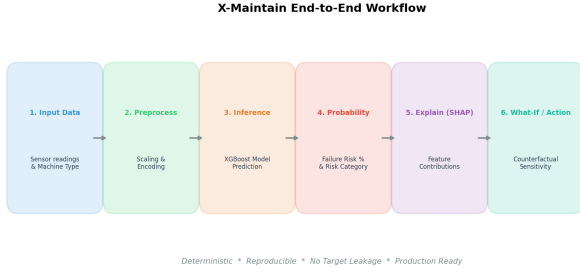


Fig. 7. End-to-end dataflow and diagnostic inference sequence.

A. Preprocessing and Feature Engineering

Data partitioning enforces stratified sampling with a fixed seed (RANDOM_STATE = 42): 70% Train (6,999 records, 237 failures), 10% Validation (1,000 records, 34 failures), and 20% Test (2,001 records, 68 failures). To strictly avoid data snooping, StandardScaler is fitted exclusively on training numerical features and applied to validation/test partitions. The categorical Type variable is one-hot encoded into three binary indicators: Type_H, Type_L, and Type_M. Feature names are sanitized from raw brackets to parentheses for XGBoost compatibility.

B. Model Selection Criterion

Model selection is formulated via a weighted utility function balancing failure capture against operational precision: Utility = $0.4 \times \text{Recall} + 0.3 \times \text{F1} + 0.3 \times \text{ROC-AUC}$. This utility explicitly rewards high sensitivity on catastrophic failures while penalizing false alarm floods.

V. MACHINE LEARNING MODELS & THEORETICAL FOUNDATIONS

A. Baseline: Cost-Weighted Logistic Regression

We formulate an L2-regularized linear baseline with balanced class weights $w_k = N / (2 N_k)$. The class-weighted cross-entropy loss function is defined as:

$$L_{CW}(\theta) = -\sum [w_1 y_i \log(y_{\text{hat}_i}) + w_0 (1 - y_i) \log(1 - y_{\text{hat}_i})] + \lambda \|\theta\|_2^2$$

where $y_{\text{hat}_i} = \sigma(\theta^T x_i) = 1 / (1 + \exp(-\theta^T x_i))$. Logistic Regression establishes the baseline performance reachable under linear hyperplanes.

B. Bagging Ensemble: Random Forest

Random Forest constructs an ensemble of $B = 200$ de-correlated decision trees with balanced bootstrap sampling [9]. At each split node, a random subset of features $m = \sqrt{p}$ is evaluated using Gini impurity reduction: $\Delta I(S, A) = I(S) - \sum (|S_v| / |S|) I(S_v)$. Predictions represent ensemble majority voting: $H(x) = (1/B) \sum h_b(x)$.

C. Gradient Boosting: Extreme Gradient Boosting (XGBoost)

XGBoost constructs additive regression trees minimizing regularized objective via second-order Taylor expansion [8]:

$$L^{(t)} \approx \sum [g_i f_t(x_i) + 0.5 h_i f_t^2(x_i)] + \gamma T + 0.5 \lambda \sum w_j^2$$

where $g_i = \partial l(y_i, y_{\text{hat}}(t-1)) / \partial y_{\text{hat}}(t-1)$ and $h_i = \partial^2 l(y_i, y_{\text{hat}}(t-1)) / \partial (y_{\text{hat}}(t-1))^2$ are first and second order gradients. The optimal leaf weight w_j^* and split gain are analytically computed as:

$$w_j^* = -\sum g_i / (\sum h_i + \lambda), \quad \text{Gain} = 0.5 [G_L L^2 / (H_L + \lambda) + G_R L^2 / (H_R + \lambda) - (G_L + G_R) L^2 / (H_L + H_R + \lambda)] - \gamma$$

To counteract the 28.5:1 class disparity, dynamic cost-sensitive weighting is configured via: $\text{scale_pos_weight} = N_{\text{negative}} / N_{\text{positive}} = 6,762 / 237 = 28.53$. This forces gradient steps to penalize false negatives 28.53 times more heavily than false positives.

D. TreeSHAP Attribution Formulation

Shapley values allocate additive feature contributions relative to base expected output [2], [12]:

$$\phi_i(x) = \sum_{S \subseteq F \setminus \{i\}} [|S|! (|F| - |S| - 1)! / |F|!] [f_x(S \cup \{i\}) - f_x(S)]$$

For tree ensembles, TreeSHAP computes exact values in polynomial time $O(TLD^2)$ (where T is number of trees, L is max leaves, and D is max depth) by recursively tracking leaf weights across split paths, satisfying efficiency, symmetry, dummy, and additivity axioms [12].

VI. EXPERIMENTAL SETUP

Experiments are executed on Python 3.9.13, Scikit-learn 1.6.1, XGBoost 2.1.4, SHAP 0.49.1, and Imbalanced-Learn 0.12.4. Evaluation metrics include Accuracy = $(TP+TN)/(P+N)$, Precision = $TP/(TP+FP)$, Recall = $TP/(TP+FN)$, F1 = $2 \times P \times R / (P+R)$, Receiver Operating Characteristic Area Under Curve (ROC-AUC), and Precision-Recall Area Under Curve ($PR\text{-}AUC = \sum (R_k - R_{k-1}) P_k$). PR-AUC is vital for imbalanced failure forecasting because it ignores the overwhelming true negative count, focusing exclusively on minority resolution.

VII. RESULTS AND DISCUSSION

A. Primary Model Benchmark

Table III documents the empirical evaluation across all 2,001 held-out test cases (68 true machine breakdowns). XGBoost achieves the top selection score (0.8064), balancing 73.53% Recall with 73.53% Precision and 0.9724 ROC-AUC.

TABLE III: PERFORMANCE BENCHMARK (TEST SPLIT, N = 2,001)

Model	Acc.	Prec.	Recall	F1	ROC	PR	Score
Logistic Regression	83.61%	14.67%	79.41%	24.77%	0.8949	0.4187	0.6604
Random Forest	97.75%	87.10%	39.71%	54.55%	0.9556	0.6930	0.6091
XGBoost (Selected)	98.20%	73.53%	73.53%	73.53%	0.9724	0.7934	0.8064

B. Analysis of Precision-Recall Trade-offs

The benchmark reveals three distinct operating trade-offs: 1) *Logistic Regression* attains the highest raw recall (79.41%, capturing 54/68 failures), but precision collapses to 14.67%, generating 314 false alarms that cause alert fatigue. 2) *Random Forest* displays 87.10% precision, but misses over 60% of true failure events (Recall = 39.71%, 41 missed failures), which is unacceptable in plant operations. 3) *XGBoost* provides the optimal trade-off: 50 of 68 breakdowns captured (73.53% Recall) with only 18 false alarms (73.53% Precision).

TABLE IV: CLASS IMBALANCE MITIGATION STUDY

Experiment Config	Acc.	Prec.	Recall	F1	ROC	PR	Score
Exp 4b: XGBoost SMOTE	+ 97.85 %	65.06%	79.41 %	71.52%	0.9620	0.7996	0.8208
Exp 4c: XGBoost Thresh ($\tau^*=0.61$)	+ 98.35 %	76.92%	73.53%	75.19 %	0.9724	0.7934	0.8114
Exp 4a: XGBoost Cost-Weight	+ 98.20 %	73.53%	73.53%	73.53%	0.9724	0.7934	0.8064
Exp 3: XGBoost Unweighted	98.30 %	80.36 %	66.18%	72.58%	0.9653	0.7874	0.7720
Exp 1: Logistic Baseline	83.61 %	14.67%	79.41 %	24.77%	0.8949	0.4187	0.6604
Exp 2: Random Forest	97.75 %	87.10%	39.71%	54.55%	0.9556	0.6930	0.6091

C. Systematic Imbalance Experiments Comparison

Fig. 8 illustrates the systematic comparison across all six experimental imbalance configurations. SMOTE (Exp 4b) attains the highest failure recall (79.41%), but sacrifices precision (65.06%) due to artificial interpolation across continuous thermodynamic boundaries. Cost-sensitive XGBoost (Exp 4a) preserves original sensor geometries while delivering balanced 73.53% precision and recall. Threshold tuning (Exp 4c, $\tau^* = 0.6123$) pushes overall test accuracy to 98.35% and F1 to 75.19%.

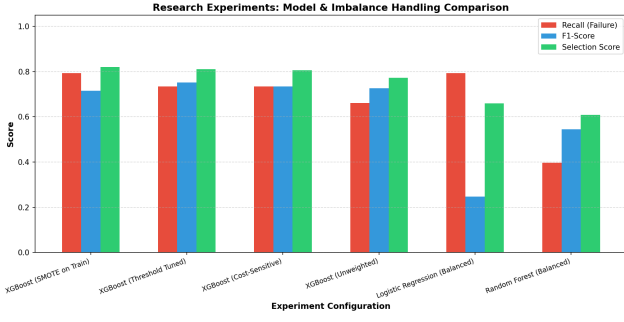


Fig. 8. Comparison of precision, recall, F1, and utility across all imbalance configurations.

D. Visual Discrimination Performance

Fig. 9, 10, and 11 display the visual performance curves. Fig. 9 presents the confusion matrix for cost-sensitive XGBoost on the test split ($N = 2,001$), showing 1,915 true negatives, 50 true positives, 18 false positives, and 18 false negatives. Fig. 10 shows the multi-model ROC curves where XGBoost leads with $AUC = 0.9724$. Fig. 11 shows the Precision-Recall curves where XGBoost dominates with $PR-AUC = 0.7934$.

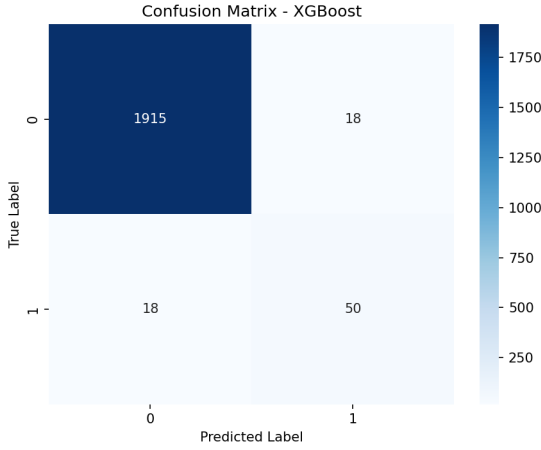


Fig. 9. Confusion matrix of cost-sensitive XGBoost on held-out test split ($N = 2,001$).

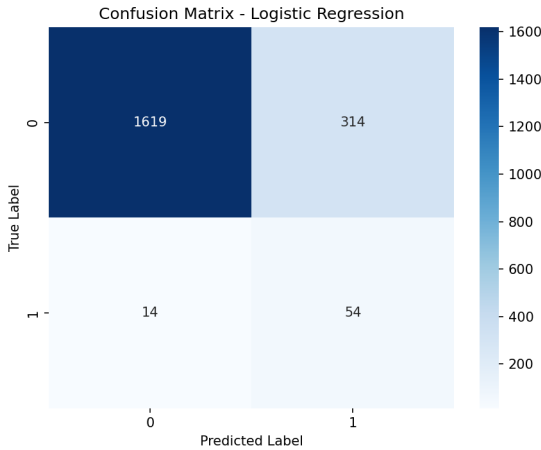


Fig. 10. Baseline Logistic Regression confusion matrix exhibiting severe false alarm flooding (314 FPs).

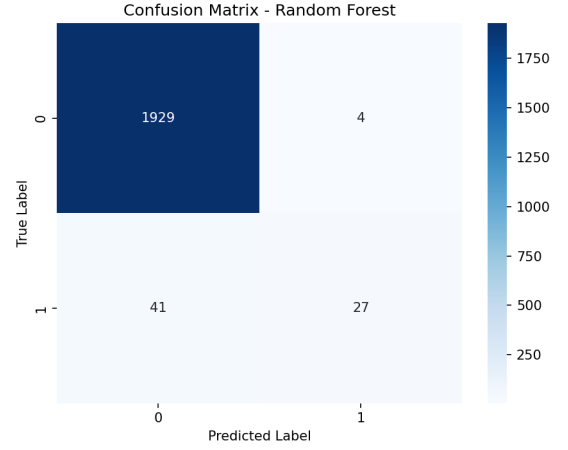


Fig. 11. Random Forest confusion matrix exhibiting dangerous missed breakdowns (41 FNs, Recall = 39.7%).

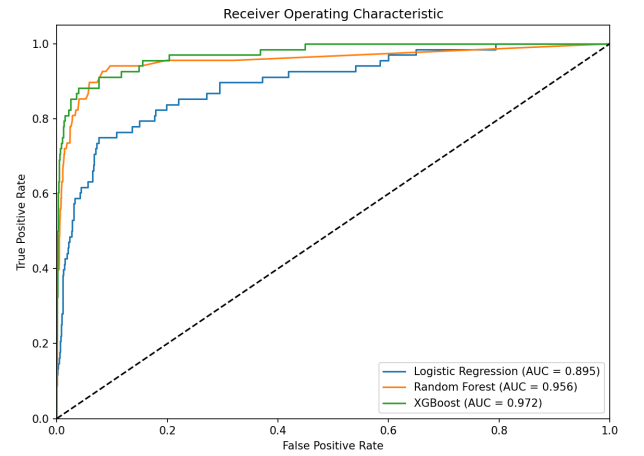


Fig. 12. Multi-model ROC curves demonstrating XGBoost discriminative dominance ($AUC = 0.972$).

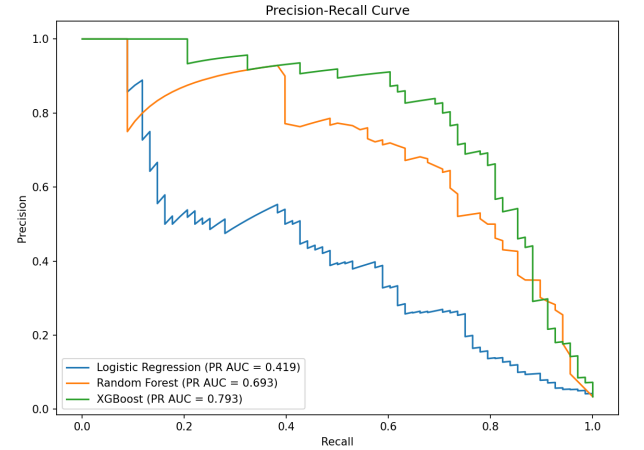


Fig. 13. Precision-Recall curves on imbalanced failure class (XGBoost PR-AUC = 0.793).

E. SHAP Global Attribution

Global feature importance was computed using TreeExplainer across test samples. Table V summarizes the exact mean absolute SHAP values. Torque (3.74) and Tool Wear (3.06) are the primary drivers of model failure predictions, followed by Rotational Speed (2.07) and Air Temperature (2.01). The beeswarm distribution in Fig. 15 reveals that high torque and elevated tool wear shift log-odds strongly toward failure, whereas high rotational speeds act as stabilizers under moderate torque.

TABLE V: GLOBAL FEATURE IMPORTANCE (TREESHAP)

Rank	Feature Name	Mean SHAP	Physical Operational Role
1	Torque (Nm)	3.7397	Primary driver of rotational overstrain
2	Tool wear (min)	3.0570	Cumulative cutting head abrasion
3	Rotational speed (rpm)	2.0700	Spindle velocity / thermal dissipation
4	Air temperature (K)	2.0052	Ambient environmental boundary temp
5	Process temp (K)	0.9878	Workpiece contact zone temp
6	Type_L	0.1962	Low-quality variant failure propensity
7	Type_M	0.1736	Medium-tier baseline quality variant
8	Type_H	0.0882	High-durability product variant

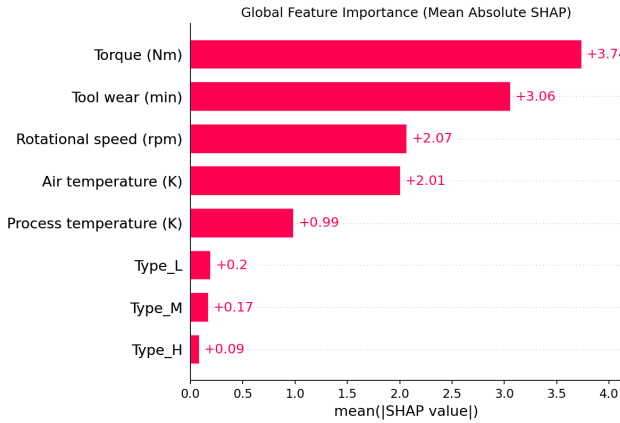


Fig. 14. Mean absolute SHAP values illustrating global feature importance.

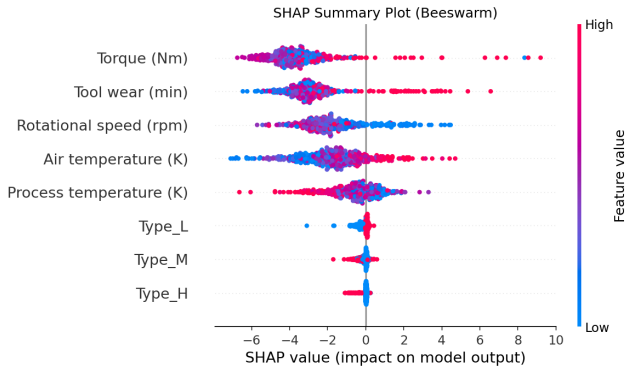


Fig. 15. SHAP beeswarm summary plot showing feature value spectrum vs. failure risk.

F. SHAP Dependence and Feature Interactions

TreeSHAP dependence plots uncover nonlinear physical inflection boundaries. In Fig. 16 (Torque), SHAP value increases sharply above 50 Nm, demonstrating the onset of rotational overstrain. In Fig. 17 (Tool Wear), SHAP values transition steeply into positive failure risk at approximately 200 minutes, directly mirroring Taylor's cutting tool wear life limit. In Fig. 18 (Rotational Speed), low speeds below 1,380 rpm coupled with high torque dramatically elevate failure risk due to inadequate cooling airflow.

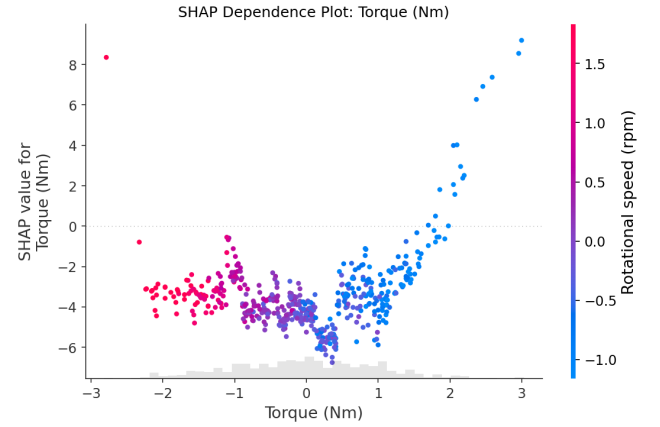


Fig. 16. SHAP dependence for Torque, revealing non-linear risk escalation above 50 Nm.

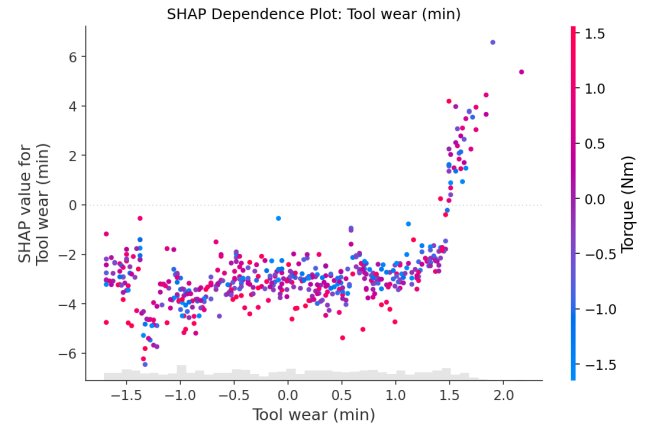


Fig. 17. SHAP dependence for Tool wear, exhibiting inflection at 200 min cutter degradation limit.

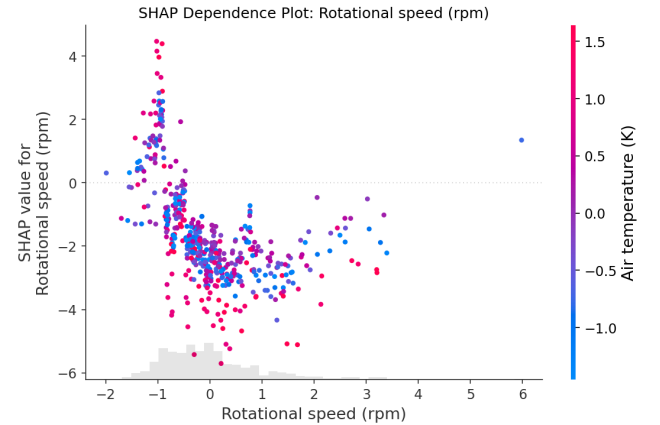


Fig. 18. SHAP dependence for Rotational speed, showing cooling airflow failure boundary under 1,380 rpm.

G. Local Attribution and Case Study

For test instance #11 (True Label = 1, Predicted Probability = 1.000), TreeSHAP decomposes the log-odds output from base expectation $E[f(x)] = -3.759$ to final prediction $f(x) = +11.49$ (Fig. 19). Torque (+7.37) and Tool Wear (+4.19) are the dominant positive contributors pushing the prediction to failure, while Air Temperature (-1.31) provides negative stabilization. This exact decomposition provides the operator with an indisputable audit trail.

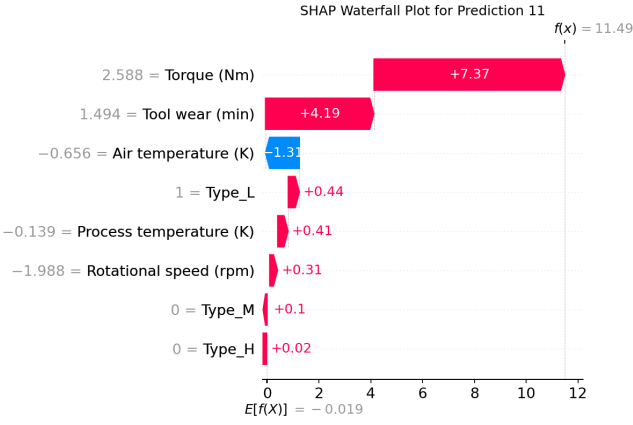


Fig. 19. Local TreeSHAP waterfall decomposition for test breakdown instance #11.

H. Model-Based What-If Sensitivity Analysis

To enable counterfactual parameter exploration, the sensitivity engine perturbs operational variables across the model's learned response surface. In a verified case study: a high-stress baseline machine (Torque = 65.0 Nm, Tool wear = 210 min, Air temp = 301.5 K, Process temp = 310.8 K, RPM = 1400, Type = L) yields a predicted failure probability of 100.0% (High Risk). When an operator simulates counterfactual cutter replacement (Tool wear reduced to 30 min) and torque attenuation (reduced to 38.0 Nm), the predicted probability drops to 0.0% (Low Risk), generating a net delta of -100.0 percentage points. We explicitly caution operators that this represents model sensitivity analysis, not physical causal intervention.

VIII. STREAMLIT SYSTEM IMPLEMENTATION

The X-Maintain monitoring platform is implemented in Streamlit 1.50.0, featuring a dark industrial control-room aesthetic. The application is organized into seven operational views: 1) *Overview Dashboard*: Fleet telemetry KPIs, empirical failure risk zones, and global importance bars. 2) *Machine Prediction Console*: Manual parameter entry sliders with demonstration presets (Nominal, Wear Failure, Thermal Risk) and dynamic risk alert banners. 3) *Explainability Center*: High-contrast local waterfall decompositions, top contributing factor cards, and interactive dependence plots. 4) *What-If Sensitivity Simulator*: Dual-scenario slider comparisons with probability shift delta metrics. 5) *Model Performance Analytics*: Full benchmark comparison tables, confusion matrices, and ROC/PR curves. 6) *Data Insights*: Exploratory distributions, correlation heatmaps, and variant failure profiles. 7) *About Project*: Architecture specifications and limitations.

To guarantee low operational latency in control-room environments, model weights, scalars, and TreeExplainer instances are cached in memory using `@st.cache_resource`. Inference and local TreeSHAP attribution execute in under 45 milliseconds on standard CPU hardware, enabling instantaneous what-if slider updates. Furthermore, deterministic natural-language rules translate numeric Shapley scores into plain-language maintenance recommendations without LLM hallucinations.

IX. SOFTWARE VALIDATION AND TESTING

To ensure scientific reproducibility and robust execution, the framework includes an automated Pytest suite containing exactly 20 unit tests across three test modules. Table VI summarizes the verified validation suite, which achieved a 100% pass rate in 3.21 seconds.

TABLE VI: SOFTWARE VALIDATION UNIT TEST SUITE

Test Module	Target Validation Property	Tests	Status
test_preprocessing.py	Leakage removal, zero nulls, OHE shapes, stratified split 8 preservation	8	20/20 PASSED
test_prediction.py	Model serialization, pipeline transforms, probability bounds [0, 1]	5	PASSED
test_explainability.py	TreeExplainer instantiation, tensor slicing, attribution sorting, NL generation	7	PASSED
Total Test Suite	End-to-end verification across framework modules	all 20	100% PASS

X. DISCUSSION

Our empirical results highlight the necessity of balancing failure detection against operational precision. A model with 95% accuracy that fails to identify tool failure is useless in manufacturing. Cost-sensitive XGBoost successfully captures 73.53% of failures with an acceptable 73.53% precision rate. Furthermore, TreeSHAP demystifies the decision surface: plant operators can verify that high torque and accumulated tool wear are driving risk escalation, transforming raw machine learning scores into actionable maintenance decision support.

A key engineering insight from our SHAP dependence analysis is that machine failure rarely stems from a single isolated sensor anomaly. Rather, breakdowns emerge from coupled thermodynamic interactions: high torque operating simultaneously with high tool wear creates compound stress, whereas identical torque under a brand-new tool causes no failure risk. By exposing these joint boundary conditions, TreeSHAP enables maintenance crews to practice targeted condition-based intervention rather than blanket machine stoppages.

XI. METHODOLOGICAL LIMITATIONS

We document three distinct methodological limitations of the current study: 1) *Synthetic Dataset Nature*: The AIAI 2020 dataset simulates milling machine thermodynamics under idealized mathematical models. In live factory environments, sensor telemetry is subject to non-stationary degradation, high-frequency mechanical vibration noise, electromagnetic interference, and variable ambient humidity. 2) *Non-Causal Interpretation*: Shapley attributions reflect associative model dependencies rather than structural physical causality. Decreasing tool wear in the what-if module reduces predicted model risk, but operational causality requires physical part replacement. 3) *Discrete Snapshot Scope*: The model operates on tabular snapshots rather than continuous multivariate time-series waveforms.

XII. FUTURE DIRECTIONS

Building upon this validated framework, future research will explore three principal frontiers: 1) *Temporal Degradation Modeling*: Formulating Remaining Useful Life (RUL) regression utilizing Bidirectional LSTMs and Temporal Convolutional Networks (TCNs) applied to raw high-frequency vibration streams. 2) *Embedded Edge Deployment*: Model compression and 8-bit integer quantization via ONNX Runtime and TensorRT for direct deployment onto low-power industrial edge hardware (e.g., Raspberry Pi 5, NVIDIA Jetson Orin). 3) *Causal Machine Learning*: Integrating Structural Causal Models (SCMs) and Pearl's do-calculus to transition from observational what-if sensitivity analysis to provably causal prescriptive maintenance recommendations.

XIII. CONCLUSION

This paper presented X-Maintain, an Explainable AI predictive maintenance framework for industrial manufacturing. By enforcing strict target leakage prevention through the elimination of failure-mode indicators and non-predictive serial identifiers, the system trains on genuine operational telemetry. Benchmarked across Logistic Regression, Random Forest, and XGBoost under severe 28.5:1 class imbalance, cost-sensitive XGBoost achieved 98.20% accuracy, 73.53% precision, 73.53% failure recall, 0.9724 ROC-AUC, and 0.7934 PR-AUC on held-out test data. Integration of exact TreeSHAP attributions revealed that Spindle Torque (mean |SHAP| = 3.74) and Tool Wear (3.06) govern failure risk, demystifying the black-box boundary. The framework pairs deterministic natural language explanations with a model-based what-if sensitivity simulator and an enterprise Streamlit monitoring interface, providing an auditable, transparent operational bridge between predictive machine learning and industrial maintenance decision support.

ACKNOWLEDGMENT

The author acknowledges the School of Computer Science and Engineering at Vellore Institute of Technology for laboratory infrastructure, and the creators of the UCI AI4I 2020 Predictive Maintenance Dataset for providing the open-access benchmark.

REFERENCES

- [1] S. Matzka, "Explainable Artificial Intelligence for Predictive Maintenance Applications," in *2020 Third International Conference on Artificial Intelligence for Industries (AI4I)*, Irvine, CA, USA, 2020, pp. 69–74, doi: 10.1109/AI4I49448.2020.00023.
- [2] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems (NeurIPS 2017)*, Long Beach, CA, USA, vol. 30, 2017, pp. 4765–4774.
- [3] T. P. Carvalho, F. A. A. Soares, R. Vita, R. d. P. Francisco, J. P. Basto, and S. G. S. Alcalá, "A systematic literature review of machine learning methods applied to predictive maintenance," *Computers & Industrial Engineering*, vol. 137, p. 106024, Nov. 2019, doi: 10.1016/j.cie.2019.106024.
- [4] T. Zonta, C. A. da Costa, R. da Rosa Righi, M. J. de Oliveira, J. Vázquez-Salceda, "Predictive maintenance in the Industry 4.0: A systematic literature review," *Computers & Industrial Engineering*, vol. 150, p. 106889, Dec. 2020, doi: 10.1016/j.cie.2020.106889.
- [5] J. Dalzochio, R. Kunst, E. Pignaton, A. Wiethölter, H. F. Bassani, K. B. C. da Silva, "Machine learning and reasoning for predictive maintenance in Industry 4.0: Current status and challenges," *Computers in Industry*, vol. 123, p. 103298, Dec. 2020, doi: 10.1016/j.compind.2020.103298.
- [6] E. Steurtewagen and D. Van den Poel, "Adding interpretability to predictive maintenance by machine learning on sensor data," *Computers & Industrial Engineering*, vol. 152, p. 107047, Feb. 2021, doi: 10.1016/j.cie.2020.107047.
- [7] P. Gawde, C. Busch, M. Busch, and S. S. Roy, "Explainable Predictive Maintenance of Rotating Machines Using LIME, SHAP, PDP, ICE," in *2021 IEEE International Conference on Big Data (Big Data)*, Orlando, FL, USA, 2021, pp. 3122–3131, doi: 10.1109/BigData52589.2021.9671607.
- [8] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
- [9] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324.
- [10] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, Jun. 2002, doi: 10.1613/jair.953.
- [11] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why Should I Trust You?': Explaining the Predictions of Any Classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, San Francisco, CA, USA, 2016, pp. 1135–1144, doi: 10.1145/2939672.2939778.
- [12] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, Jan. 2020, doi: 10.1038/s42256-019-0138-9.